

Tabla de contenido

1. Resumen Ejecutivo	1
2. Stack Tecnológico	1
3. Arquitectura de Software	1
A. Capa de Presentación (Views)	1
B. Capa de Lógica de Negocio (Controllers)	2
C. Capa de Datos (Models & Data)	2
4. Flujos Principales de la Solución	2
4.1. Flujo de Autenticación y Registro	2
4.2. Motor del Juego (Ciclo de Vida)	2

1. Resumen Ejecutivo

WhoWant2B es una plataforma web de trivias inspirada en el formato "Quién quiere ser millonario". La solución permite la gestión de un banco de preguntas categorizadas por niveles de complejidad y la ejecución de partidas en tiempo real para usuarios registrados.

2. Stack Tecnológico

- Framework: ASP.NET Core MVC.
- Persistencia: Entity Framework Core (SQL Server probable).
- Frontend: Razor Pages, CSS3 (Custom), Bootstrap 5, jQuery.
- Seguridad: SHA-512 para hashing de credenciales.
- Gestión de Estado: Session State y parámetros de ruta.

3. Arquitectura de Software

La aplicación utiliza una arquitectura de N-Capas simplificada dentro del patrón MVC:

A. Capa de Presentación (Views)



- Vistas de Juego: Interfaces dinámicas que cambian según el estado de la partida (Registro, Pregunta, Derrota, Retiro).
- Vistas de Configuración: Dashboards administrativos para la gestión de datos maestros (CRUD de Categorías y Preguntas).

B. Capa de Lógica de Negocio (Controllers)

- JuegoController: Actúa como el motor del juego. Gestiona la lógica de progresión (subida de nivel cada 3 aciertos), la selección aleatoria de preguntas no repetidas y el proceso de autenticación/registro.
- ConfiguracionController: Implementa la lógica de administración, gestión de transacciones para la creación atómica de preguntas y sus opciones, y paginación de registros.

C. Capa de Datos (Models & Data)

- ApplicationDbContext: El mediador entre los objetos C# y la base de datos.
- Modelos de Entidad: Usuario, Pregunta, Opcion, Categoria, Complejidad e Historico.
- ViewModels: Uso de PreguntaConOpciones_viewModel para desacoplar la vista de la estructura pura de la base de datos en formularios complejos.

4. Flujos Principales de la Solución

4.1. Flujo de Autenticación y Registro

El sistema implementa un "Login Automático". Si un usuario no existe por su nombre real, se le genera un login basado en timestamp y se guarda su contraseña con un hash SHA-512. Si existe, se valida el hash de forma estructural.

4.2. Motor del Juego (Ciclo de Vida)

1. Selección: Se filtran preguntas por IdComplejidad y se excluyen las idsRespondidas.
2. Aleatoriedad: Se utiliza `OrderBy(p => Guid.NewGuid())` para garantizar que cada partida sea distinta.
3. Validación: El método `ValidarRespuesta` incrementa el puntaje y gestiona el salto de complejidad (Nivel 1 al 5).
4. Persistencia de Logros: Al retirarse o perder, se registra un Historico con los puntos y el estado final del juego.
5. Diseño de Base de Datos (Relacional)
 - Categorías (1:N) Preguntas: Clasificación temática.
 - Complejidades (1:N) Preguntas: 5 niveles de dificultad.

- Preguntas (1:N) Opciones: Cada pregunta tiene exactamente 4 opciones, donde solo una es Valida.
- Usuarios (1:N) Historico: Seguimiento de desempeño.



6. Consideraciones de Calidad y Seguridad

- Transaccionalidad: El sistema utiliza IDbContextTransaction en la creación de preguntas para asegurar que no existan preguntas sin opciones (integridad referencial).
- Seguridad de Datos: Las contraseñas no se almacenan en texto plano, sino como arreglos de bytes (Binary) tras el proceso de hashing.
- UX/UI: Uso de Clip-path CSS para emular la estética hexagonal característica de los programas de trivia televisivos.